Smart Digit Reader

Handwritten Digit Recognition System

Chrishandra Lewich

Tier 1 Project

Problem

Organizations still rely on handwritten forms, receipts, and notes.

Manually reading and typing digits is slow and error-prone.

A simple automated system can reduce errors and save time.

Solution

Smart Digit Reader automatically identifies handwritten digits (0–9) from images.

The system takes an image → predicts the digit → returns the result.

A lightweight digit-recognition system using image classification.

Technical Approach

- Computer vision task: **Image classification**
- Model: **Small CNN** or pretrained MNIST model
- Framework: **PyTorch**
- Reasoning: MNIST is simple, clean, and works well with lightweight models.

Data Plan

- Dataset: **MNIST** (public, 70,000 labeled images)
- Image type: 28×28 grayscale handwritten digits
- Labels: 10 classes (0–9)
- No data collection or annotation required
- Split: 60k training, 10k testing

System Diagram

[Input Image]

↓

[Preprocessing]

↓

[CNN Model]

↓

[Predicted Digit]

Success Metrics

- Primary metric: **Accuracy**
- Target: ~98% accuracy on the test set
- Secondary metrics:
  - Fast inference
  - Small model size

Week-by-Week Plan

- **Week 10:** Dataset + environment setup
- **Week 11:** Train or fine-tune model
- **Week 12:** Test and improve accuracy
- **Week 13:** Build demo + short video
- **Week 14:** Final testing + documentation
- **Week 15:** Present project

Challenges/ Backup Plan

- Low accuracy → add simple augmentation (rotate, shift)
- Training errors → use a pretrained MNIST model
- Time constraints → train on a smaller subset first

Resources Needed

- Compute: Google Colab
- Tools: PyTorch
- Cost: $0
- Deliverables: GitHub repo + slide deck

Model Architercture (CNN Overview)

- Input: 28×28 grayscale image
- Layers:
- Conv2D → ReLU → MaxPool
- Conv2D → ReLU → MaxPool
- Flatten → Fully Connected → Softmax
- Output: 10 classes (digits 0–9)
- Lightweight design for fast inference
- Framework: PyTorch

Training & Evaluation Results

- Dataset: MNIST (60k train / 10k test)
- Training accuracy: ~99%
- Test accuracy: ~98%
- Loss curve shows stable convergence
- Confusion matrix confirms strong digit separation
- Inference time: <0.1s per image

System Demo

- Input: Handwritten digit image
- Output: Predicted digit displayed instantly
- Example: Input “7” → Output “Predicted: 7”
- Interface: Simple notebook cell or web form
- Demonstrates Smart Digit Reader’s speed and accuracy

Limitations & Future Improvements

- Limited to single digits (0–9)
- Works best on clean MNIST-style images
- Future goals:
- Extend to multi-digit recognition
- Add real-world handwriting dataset
- Deploy as web or mobile app
- Integrate accessibility features (voice feedback)

Conclusion & Reflection

- Achieved ~98% accuracy with lightweight CNN
- Demonstrated automated digit recognition
- Reduced manual data entry errors
- Ethical reflection: promotes accessibility, minimal bias
- Deliverables: GitHub repo + demo video + presentation
- Smart Digit Reader — simple, effective, and scalable